Shader Documentation

This project includes two shader effects, the primary effect is a collection of shaders that emulate the rendering techniques used by games that ran on the Playstation 1 (PS1). The secondary effect is a toon lighting and shading model that quantizes light attenuation, and has support for multiple lights. The effects are shown off across two scenes, a lighter day scene, and a darker rain scene.

Table of Contents

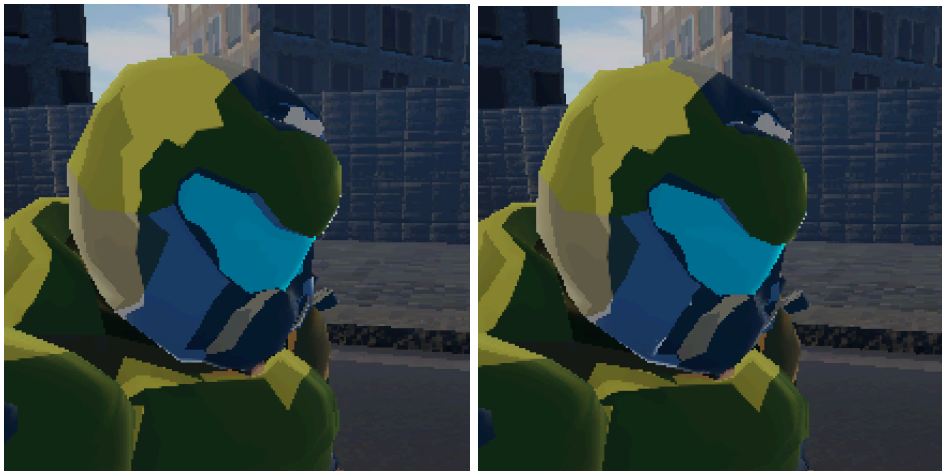
Table of Contents **1**
Playstation effects **2**
Vertex snapping 2
Affine texture mapping 3
Controls 4
Screen-space effects **5**
Colour quantization 5
Dithering 6
Low resolution 7
Controls 8
Toon shading **8**
Implementation 8
Controls 9
End user **9**
Performance **10**
Scene controls **11**
Character controller 11
Shader options menu 11
Interesting Details **12**
Resources **12**
Asset credits **12**

Playstation effects

Playstation effects can be observed by using a material instance of either “PSX Material” or “PSX Material Toon”.

Vertex snapping

Modern rendering engines use floating point values for the positions of vertices, which allows for sub-pixel rasterization. However the PS1 instead used integer values. This meant that sub-pixel rasterization was not possible on the PS1 as vertices would snap to the nearest pixel.



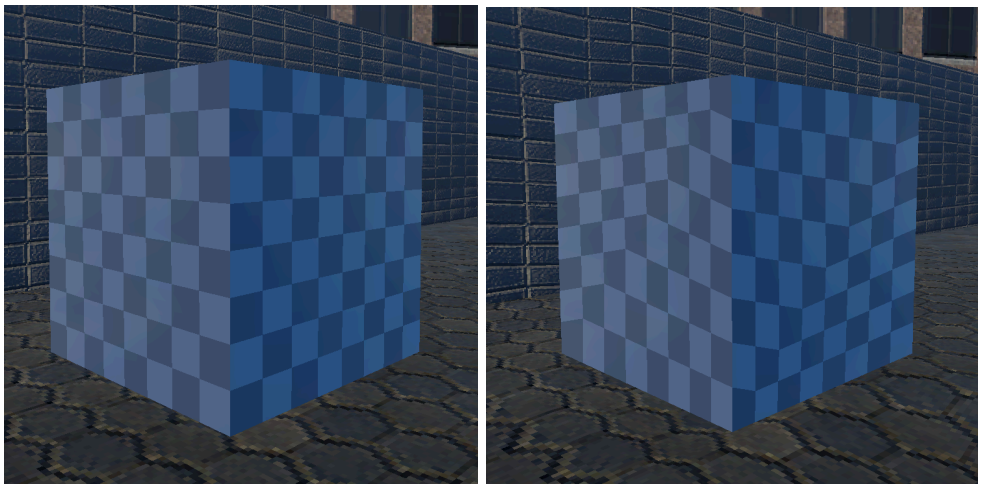
- Vertex snapping disabled (left)
- Vertex snapping enabled (right)

NOTE: This effect is best observed in motion rather than a static image.

This effect is achieved by flooring a vertex’s position in view space after scaling the position by a ‘pixel resolution’ value.

Affine texture mapping

When projecting textures, modern rendering engines use calculations that are perspective correct. This was not the case for the PS1, the calculations for the PS1 did not correct for perspective, resulting in textures appearing warped or skewed.

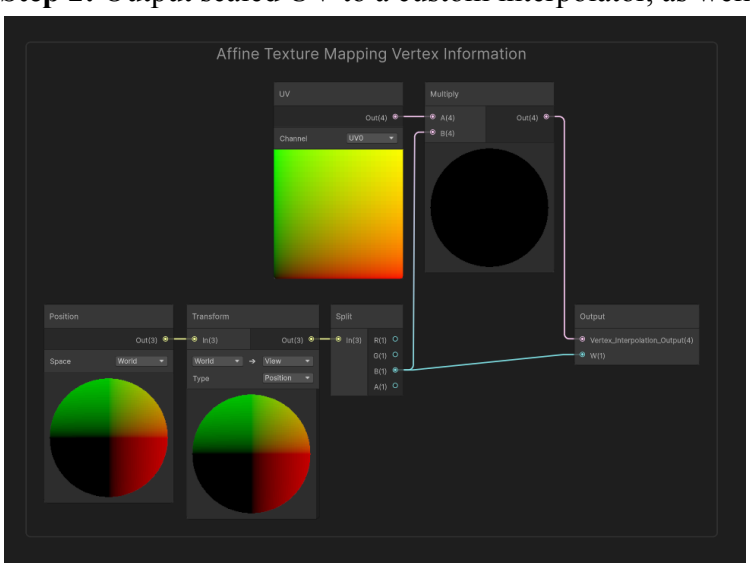


- Perspective correct mapping (left)
- Affine mapping (right)

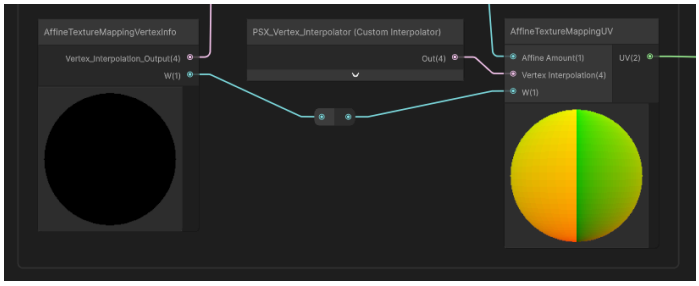
To emulate this characteristic of the PS1 we purposefully undo this perspective correction. To do so in Unity's shader graph…

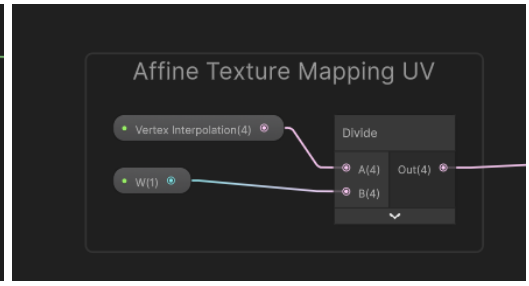
Step 1: In the vertex shader multiply the texture UV's by the 'W' component of a position in view space.

Step 2: Output scaled UV to a custom interpolator, as well as the 'W' component.



Step 3: Divide every value by W in the fragment shader





Step 4: A modern convenience was added to allow the user to define the amount of affine texture mapping they wish to use. This can be changed on a per material basis.

Controls

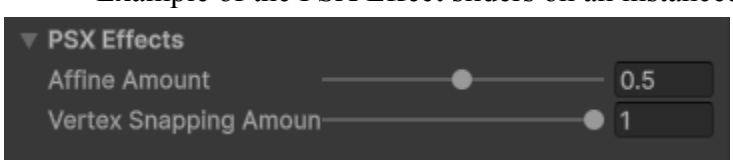
Vertex snapping and affine texture mapping logic is kept within their respective shader subgraphs. These subgraphs are then utilized in the 'PSX Material' and 'PSX Material Toon' shader graphs. Both of these effects can be either toggled on/off in the global shader uniforms, or lerped with a 0-1 value on a per material basis.

- Code for toggling vertex snapping and affine texture mapping on/off. (GameManager.cs)

```
// Enable snap vertices
snapVertices = true;
Shader.SetGlobalInteger("_Snap_Vertices", snapVertices ? 1 : 0);

// Enable affine texture mapping
affineTextureMapping = true;
Shader.SetGlobalInteger("_Affine_Texture_Mapping", affineTextureMapping ? 1 : 0);
```

- Example of the PSX Effect sliders on an instanced material



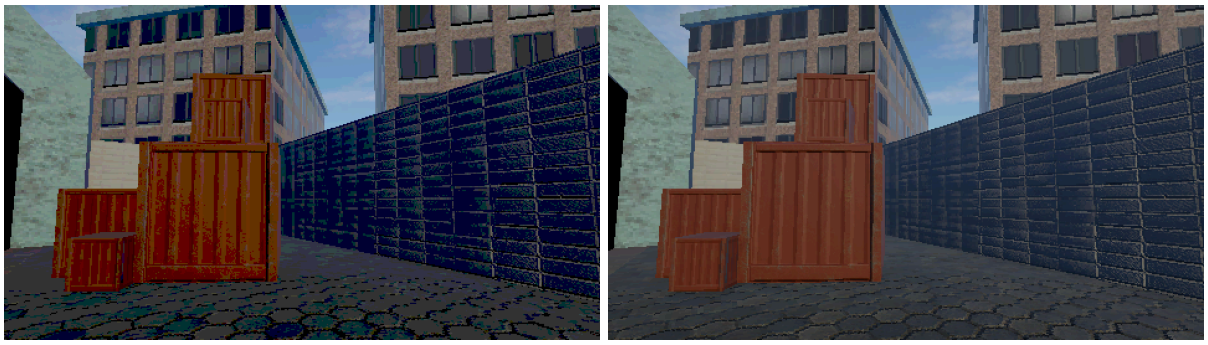
Screen-space effects

Colour quantization

Due to memory limitations imposed by the less powerful hardware of the PS1. Colours were 5-bit instead of 8-bit. A 24-bit colour, (one that uses 8 bits per RGB colour channel,) can represent up to 16,777,216 unique colours, whereas a 15-bit colour (5 bits per channel) can only represent up to 32,768 unique colours.

- Unique 8-bit colours = (2^8 bits)^3 channels = 16,777,216.
- Unique 5-bit colours = (2^5 bits)^3 channels = 32,768.

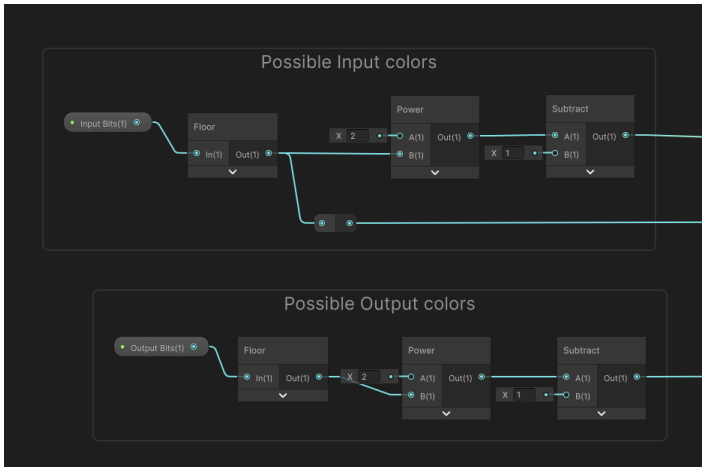
Modern displays are capable of displaying 8-bit colours so to emulate the 5-bit colours on the PS1 colour quantization is used. This is achieved by remapping the range of 8-bit input values to a range of 5-bit output values.



- Colour quantization enabled (left)
- Colour quantization disabled (right)

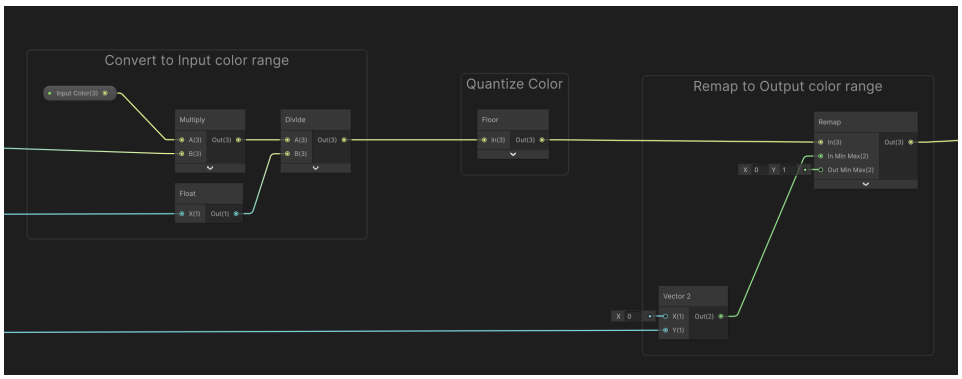
To emulate this characteristic of the PS1 in Unity's shader graph…

Step 1: Calculate range of possible values for both the number of input bits and output bits.



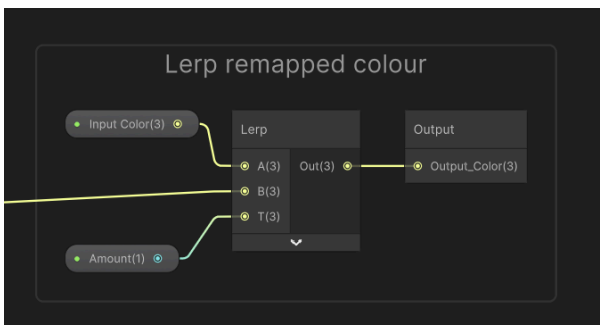
Step 2, 3: Remap input colour to ranges and quantize colour.

- Quantization is achieved by taking the floor of the value after it has been multiplied into the input color range. Extra colour detail is purposefully lost in this step.



Step 4: A modern convenience was added to allow the user to define the amount of colour quantization they wish to use.

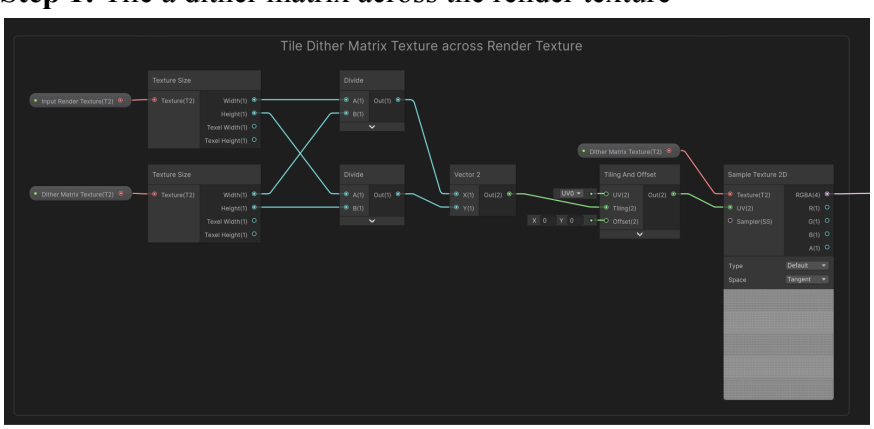
- This was done by lerping the input colour to the remapped colour with a 0-1 value.



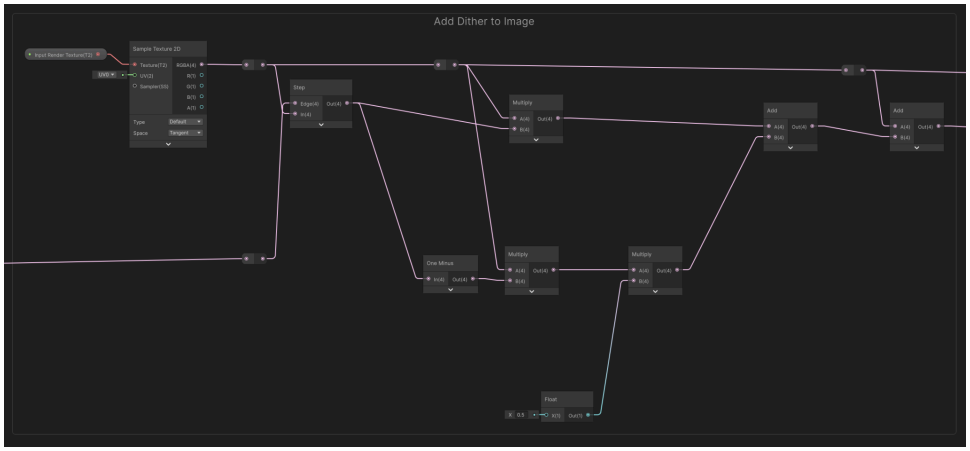
Dithering

Due to intense colour banding as a result of colour quantization, a layer of dithering is added to reduce this effect.

Step 1: Tile a dither matrix across the render texture

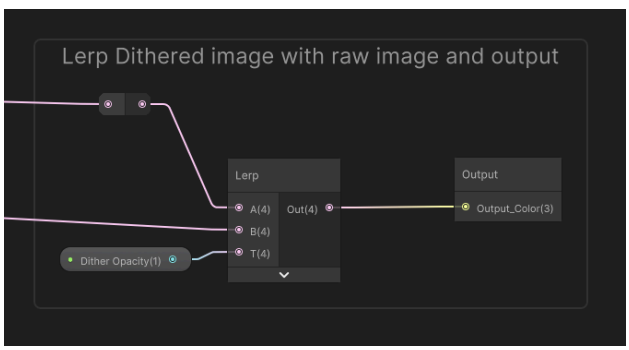


Step 2: Add the value of the dither matrix to the image.



Step 3: A modern convenience was added to allow the user to define the amount of dithering they wish to use.

- This was done by lerping the input colour to the dithered colour with a 0-1 value.



Low resolution

The maximum resolution that the PS1 was capable of outputting was ‘640x480’ (4:3 aspect ratio). However, many games would use a lower resolution than that for performance. Most PS1 games would output their resolution at or around ‘320x240’ (4:3 A.R). Modern day games and displays usually have a resolution of at least ‘1920x1080’ (16:9 A.R).

To emulate the lower resolution but still respect the aspect ratio of modern displays, The scene camera outputs to a render texture with a resolution of ‘480x270’ (16:9 A.R). Unity then displays this render texture as a UI Image element that matches the screen resolution.

Controls

NOTE: To allow changing these values easily at runtime for demonstration, the following values have been made as global uniforms. For a real games context these should instead be “Per Material” as it is unlikely that these values would have to change during normal gameplay.

Dithering and colour quantization logic is kept within their respective shader subgraphs. These subgraphs are then utilized in the ‘PSX Screen Effects’ shader graph. Both of these effects can be either toggled on/off, or lerped with a 0-1 value in the global shader uniforms.

- Code for toggling dithering and colour quantization on/off. (GameManager.cs)

```
dither = true;
Shader.SetGlobalInteger("_Dither", dither ? 1 : 0);

colorQuantization = true;
Shader.SetGlobalInteger("_Color_Quantization", colorQuantization ? 1 : 0);
```

- Code for setting lerp amounts for Dithering and colour quantization. (GameManager.cs)

```
Shader.SetGlobalFloat("_Color_Quantization_Amount", 0.5f);
Shader.SetGlobalFloat("_Dither_Amount", 0.25f);
```

Toon shading

A toon lighting shader was made alongside the play station effects shader. These effects are combined in the ‘PSX Material Toon’ shader graph.

Implementation

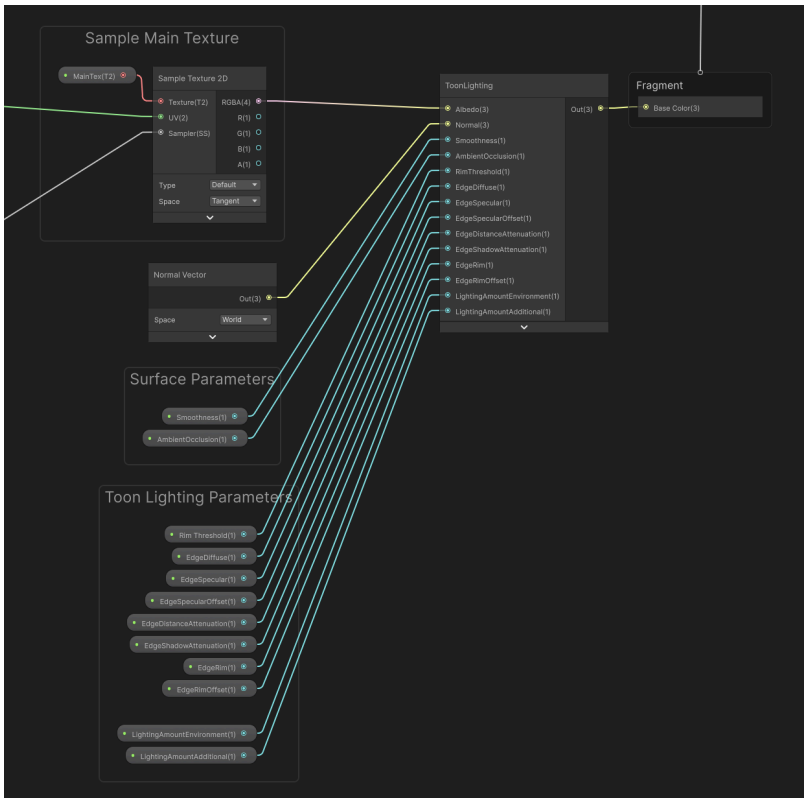
Toon shading is achieved with a custom lighting pass in a HLSL shader (ToonLighting.hlsl). The toon lighting starts off as an implementation of the blinn-phong lighting model where ambient, diffuse and specular components are calculated individually. To achieve the toon lighting effect, each component is put through a smoothstep function with a threshold for that specific component to be quantized by. Using smoothstep lets the user have smoother transitions between areas of ‘lit’ and ‘unlit’ by controlling where the smoothstep begins and ends.

On top of the normal lighting components included in blinn-phong shading, the toon shader also utilizes a rimlight that is based on a fresnel effect..

Controls

‘PSX Material Toon’ contains many adjustable parameters to control the thresholds and edges of the toon shading.

- **LightingAmountEnvironment** and **LightingAmountAdditional** can be used to multiply the amount of environment, or additional toon lighting with a 0-1 value. This is used to make the transition between areas of ‘lit’ and ‘unlit’ less harsh.

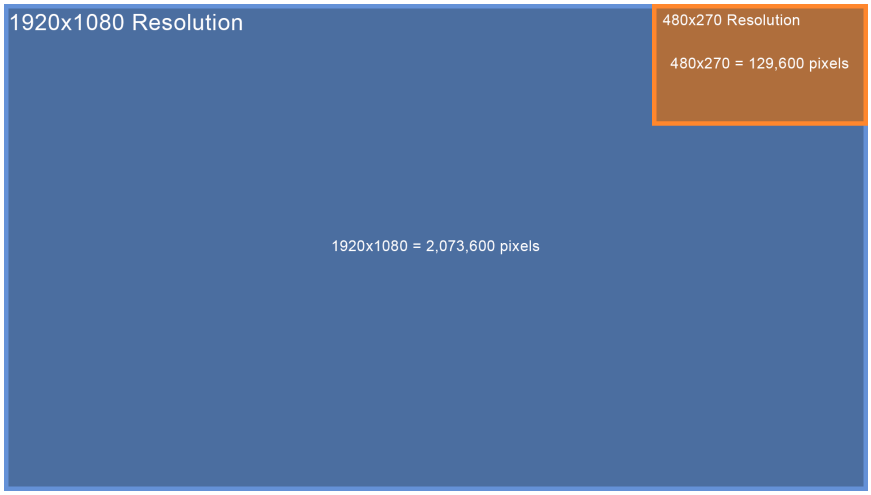


End user

Different parts of the shader have been compartmentalised by breaking effects down into either one or several subgraphs. This makes the shaders easier to understand, as well as mix and match effects for different shaders. For example, a user could quite easily add an additional screen space effect by using the existing Dithering and Colour Quantization subgraphs. This approach was also used when creating the ‘PSX Material Toon’ shader, as it uses the same ‘VertexSnapping’ and ‘AffineTextureMapping’ subgraphs as the ‘PSX Material’ shader.

Performance

Overall, the scene is very performant. In large part due to the low resolution. A modern display with a ‘1920x1080’ resolution results in 2,073,600 pixels that need to be rendered. The low resolution of ‘480x270’ results in 129,600 pixels. Only 1/16th of the original resolution.



A small potential optimisation could be made within the Dithering shader. Currently, the shader samples a texture to get the values for the dither matrix. This approach is also additive, where a higher “dither amount” makes the screen brighter.

If the dithering shader was written in code instead of being done in a shader graph, it would be possible to index a matrix type directly instead of sampling a matrix texture. This matrix type could also contain negative values instead of only positive values contained in a texture, allowing the shader to add dithering without adding brightness to the image.

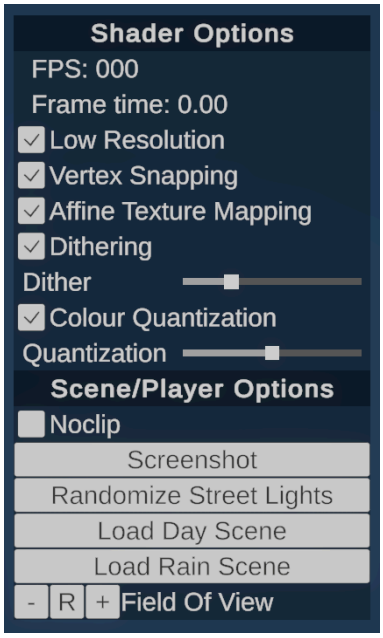
Scene controls

Each scene uses a first person character controller for the base movement, but also offers a shader options menu. Several other controls are in place as well as these core aspects. The two included scenes utilize the same shaders across both screens but with different materials. In both scenes, the floating sphere and lowpoly doomslayer mesh in the center of the scene use the ‘PSX Material Toon’ shader, every other material in the scene uses the regular ‘PSX Material’. Materials in the ‘rain’ scene typically have much higher smoothness to give the appearance of a wet surface.

Character controller

- **Move**: WASD
- **Look**: Mouse
- **Jump**: Spacebar
- **Move up** (requires noclip): Q
- **Move down** (requires noclip): E
- **Open/Close shader options menu**: R
- **Lock/Unlock mouse**: Tab
- **Toggle Flashlight**: F

Shader options menu



- **FPS** and **Frame time** are used for simple profiling.
- **Low Resolution**: Toggles the low resolution render texture being used by the camera.
- **Vertex Snapping**: Toggles vertex snapping globally.
- **Affine Texture Mapping**: Toggles affine texture mapping globally.
- **Dithering**: Toggles dithering globally, (requires 'Low Resolution' to be enabled.)
- **Dither**: Control the amount of dithering, (requires 'Dithering' to be enabled.)
- **Colour Quantization**: Toggles colour quantization globally, (requires 'Low Resolution' to be enabled.)
- **Quantization**: Control the amount of colour quantization, (requires 'Colour Quantization' to be enabled.)
- **Noclip**: Toggles flight and collision for the player
- **Screenshot**: Briefly hides the UI and takes a screenshot that gets saved to your PC.
- **Randomize Street Lights**: Randomize the colour of the two street lights in the scene.
- **Load Day/Rain Scene**: Load the respective scene.
- **Field of View**: Increase/Decrease/Reset the camera FOV.

Interesting Details

- Another key aspect of PS1 visuals are low polygon models and low resolution textures. A lot of detail will be lost in a model when using a low resolution and vertex snapping. All textures in this project are no larger than 256x256 in size.
- The toon variant of the shader makes objects easily stand out against non toon shaded environments. I found that the best use case for the toon variant of the shader was for either characters, or interactable objects that a designer wants to draw a player's attention.

Resources

- Why Did PS1 Games Look Like That? For understanding the requirements of a playstation shader and the math behind some of the effects.
- https://nedmakesgames.medium.com/creating-custom-lighting-in-unitys-shader-graph-with-universal-render-pipeline-5ad442c27276 for understanding implementation of custom shader functions in shader graphs and the basics of lighting.

Asset credits

- Low-poly Doomslayer model: Made by myself in 2020 for a Risk of Rain 2 mod that was never finished.
- All environment textures are from either CC0Textures.com or FreePBR.com
- All sound effects are from https://sonniss.com/gameaudiogdc/